Einführung

Bevor wir starten: Haben Sie schon einmal darüber nachgedacht, wie aus einem Hochsprachen-Programm ein tatsächliches Verhalten eines Computers wird? Die Idee liegt darin, die Komplexität dieser Frage in handhabbare Schichten zu unterteilen. Wie diese Schichten genau aussehen, werden wir in dieser Vorlesung systematisch erarbeiten.



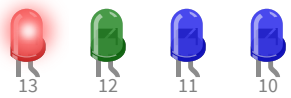
Parameter	Kursinformationen
Veranstaltung:	`Digitale Systeme / Eingebettete Systeme`
Semester:	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`Motivation der Vorlesung "Eingebettete Systeme" und Beschreibung der Organisation der Veranstaltung`
Link auf GitHub:	https://github.com/TUBAF-IfI-LiaScript/VL_EingebetteteSysteme/blob/master/00_Einfuehrung.md
Autoren:	Sebastian Zug, André Dietrich & `fjangfaragesh`, `FnHm`, `gjaeger`, `ShyFlyGuy`, `Lalelele`

Fragen an die Veranstaltung

- Welche Abstraktionsebenen durchlaufen wir vom Arduino-Code bis zur Hardware-Ebene?
- Warum lohnt sich das Verständnis eingebetteter Systeme für Informatik, Robotik und Ingenieurwesen?
- Welche Rolle spielen Mikrocontroller in modernen digitalen Systemen?
- Wie unterscheiden sich eingebettete Systeme von herkömmlichen Computersystemen?
- Welche praktischen Anwendungsgebiete finden Sie für eingebettete Systeme in Ihrem Alltag?
- Was bedeutet „Echtzeit“ im Kontext eingebetteter Systeme?
- Wie wirken sich Ressourcenbeschränkungen (Speicher, Rechenzeit, Energie) auf die Programmierung aus?

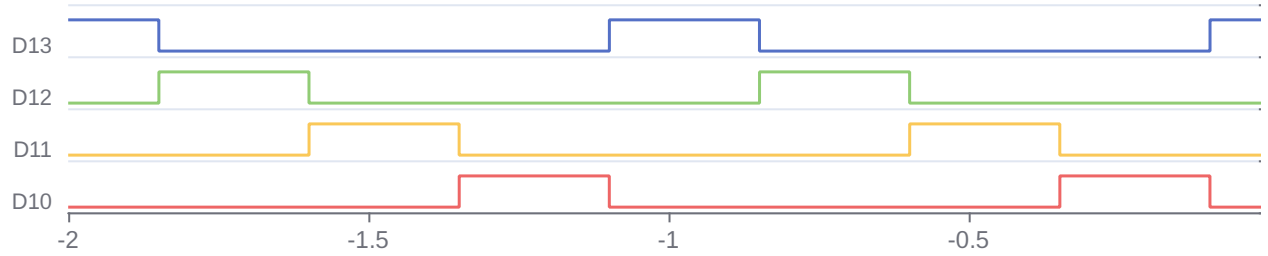
Worum geht es in dieser Vorlesung?

Starten wir mit etwas Code …



Simulation time: 00:42.120 (101%)

Pegel an den Pins 13 bis 10



BlinkLEDs.ino

```
byte leds[] = {13, 12, 11, 10};

void setup() {
  Serial.begin(115200);
  for (byte i = 0; i < sizeof(leds); i++) {
    pinMode(leds[i], OUTPUT);
  }
}

int i = 0;
void loop() {
  Serial.print("LED: ");
  Serial.println(i);
  digitalWrite(leds[i], HIGH);
  delay(250);
  digitalWrite(leds[i], LOW);
  i = (i + 1) % sizeof(leds);
}
```

```
Sketch uses 2238 bytes (6%) of program storage space. Maximum is 32256
bytes.
Global variables use 200 bytes (9%) of dynamic memory, leaving 1848
bytes for local variables. Maximum is 2048 bytes.


LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
```

```
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
```

```
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
```

```
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
LED: 2
LED: 3
LED: 0
LED: 1
```

```
LED: 2
LED: 3
LED: 0
```

> **Frage:** Welche Funktion hat der Code? Lesen Sie die Antwort im Diagramm ab, bevor Sie weiterklicken.

Der Code schaltet nacheinander vier LEDs an den Pins 13 bis 10 ein und aus, jede für 250 Millisekunden. Das Diagramm darunter zeigt dasselbe wie ein Logikanalysator: Zu jedem Zeitpunkt ist höchstens ein Pin auf HIGH. Hinter diesen knapp zwanzig Zeilen steckt viel mehr, als man sieht – Bibliothekscode der Arduino-Umgebung, ein Compiler und am Ende Transistoren, die schalten. Die Befehle können Sie unter https://www.arduino.cc/reference/en/ nachschlagen.

> **Frage:** Was ist notwendig, damit dieses Programm auf einem Mikrocontroller ausgeführt werden kann?

Der Code muss übersetzt werden. Der Compiler macht aus dem C++-Code Assemblerbefehle für die AVR-Architektur, der Assembler daraus Maschinencode – Zahlen, die der Mikrocontroller direkt ausführen kann. Der Linker fügt die benötigten Teile der Arduino-Bibliothek hinzu und legt fest, wo im Speicher alles steht. Das Ergebnis wird als Hex-Datei auf den Controller übertragen.

> Keine Sorge: Diese Zahlen müssen Sie heute nicht lesen können – es geht nur darum, wohin die Reise geht.

```
:100000000C945D000C9485000C9485000C94850084
:100010000C9485000C9485000C9485000C9485004C
:100020000C9485000C9485000C9485000C9485003C
:100030000C9485000C9485000C9485000C9485002C
:100040000C9412020C9485000C9482020C945C02B5
:100050000C9485000C9485000C9485000C9485000C
:100060000C9485000C948500000000024002700FB
:100070002A00000000000250028002B0004040404CE
:1000800004040404020202020202030303030342
:10009000010204081020408001020408102001021F
...
```

Jede Zeile hat dasselbe Format: `:` · Anzahl der Datenbytes (`10` = 16) · Adresse im Programmspeicher (`0000`) · Typ (`00` = Daten) · die Datenbytes · eine Prüfsumme.

Am Ende des Semesters können Sie jeden Schritt dieser Transformation erklären.

Schauen Sie auf die ersten vier Datenbytes: 0C 94 5D 00. Das ist ein einziger Maschinenbefehl, ein Sprungbefehl – jmp 0xBA. Im Befehl steht die Wortadresse 5D; weil jedes Befehlswort zwei Bytes lang ist, entspricht das der Byteadresse 0xBA. Der Befehl steht an Adresse 0, dort beginnt der Controller nach einem Reset. Von dort springt er zum Startcode, der das Programm vorbereitet und schließlich setup und loop aufruft. Die Grundprinzipien, die wir an diesem kleinen Controller lernen, gelten für jeden Rechner.

Was ist ein eingebettetes System?

Die Vorlesung heißt „Eingebettete Systeme“ – also klären wir den Begriff, bevor wir losgehen. Ihr Arduino mit den vier LEDs ist bereits eines.

Ein **eingebettetes System** ist ein Rechner, der fest in ein technisches Gerät eingebaut ist und dort eine bestimmte Aufgabe erfüllt – meist ohne dass man ihn als Rechner wahrnimmt. Typisch sind:

- eine **feste Aufgabe** statt beliebiger Programme (Waschmaschine, Motorsteuerung, Wetterstation),
- **Sensoren und Aktoren** statt Tastatur und Bildschirm,
- **knappe Ressourcen**: wenig Speicher, wenig Rechenleistung, wenig Energie, geringe Kosten,
- oft **Echtzeitanforderungen**: Die Reaktion muss nicht nur richtig, sondern auch rechtzeitig kommen – ein Airbag, der eine Sekunde zu spät auslöst, ist nutzlos.

Welche dieser Geräte sind eingebettete Systeme?

- [ ] die Steuerung einer Waschmaschine
- [ ] ein Herzschrittmacher
- [ ] ein Laptop, auf dem Sie Ihre Hausaufgaben schreiben
- [ ] das Steuergerät für die Bremsen im Auto

Die Reise: 16 Vorlesungen zum Verständnis

Vorlesung	Was lernen Sie hier?	Wo steckt das im Arduino?
00 Einführung	Die große Vision	vom Sketch zur Hex-Datei
01 Historie	Von Leibniz zu Arduino	warum der Chip so aussieht, wie er aussieht
02 Boolesche Algebra	0 und 1 verstehen	`if (a && b)`, Bitmasken
03 Minimierung	Effizienz in Logik	weniger Gatter für dieselbe Funktion
04 Schaltnetze	Logik wird Hardware	die Logik im Chip, inklusive ihrer Laufzeiten
05 Standardschaltnetze	Decoder & Multiplexer	Registerauswahl, Kanalwahl beim `analogRead`
06 FlipFlops	Speicher entsteht	das Bit in `DDRB`, das sich „Ausgang“ merkt
07 Schaltwerke	Zustandsmaschinen	Steuerungen mit Zuständen, z. B. eine Ampel
08 Standardschaltwerke	Counter & Register	der Zähler hinter `millis()`
09 Rechnerarithmetik	Addition in Hardware	`int`, Überlauf, `float`
10 CPU-Basis	Der erste Prozessor	Rechenwerk, Steuerwerk, Speicher
11 Modell-CPU	Befehl für Befehl	was mit `0C 94 5D 00` passiert
12 AVR-CPU	**Ihr Arduino-Chip!**	ATmega328P, Toolchain, Register
13 ADC	Analog trifft Digital	`analogRead`
14 Timer & Interrupts	Unterprogramme, Stack, Timing	`attachInterrupt`, `delay`, `analogWrite`, Echtzeit

15 Aktoren	Hardware bewegen, Ausblick	Servo, Schrittmotor, Echtzeit und Energiesparen

Für 16 Vorlesungen haben wir 21 Termine: Einige Themen bekommen zwei Termine, am Ende bleiben Zeit für Wiederholung und ein Puffer.

Was steht am Ende?

Diese Fragen klingen heute noch nach Zauberei. Am Ende des Semesters sollten Sie jede davon in wenigen Sätzen beantworten können – und genau solche Fragen finden Sie in der Klausur wieder.

Sie können diese Fragen fundiert beantworten:

1. **„Wie wird `pinMode(13, OUTPUT)` zu einer Hardware-Konfiguration?“** *Von der Arduino-Funktion über Assembler zum Bit im Register `DDRB` und zur Tri-State-Logik am Pin*
2. **„Was bedeutet `Serial.begin(115200)`?“** *Die serielle Schnittstelle (UART) überträgt Bit für Bit ohne gemeinsamen Takt – deshalb vereinbaren beide Seiten zu Beginn die „Sprechgeschwindigkeit“.*
3. **„Ist ein `delay(1000)` tatsächlich genau 16.000.000 Taktzyklen lang?“** *Taktfrequenz, Befehlszyklen und Timer-Hardware verstehen*
4. **„Was passiert im ATmega328P während `analogRead(A0)`?“** *ADC-Wandlung, Abtastung, Quantisierung und Speicherung*
5. **„Wie führt die CPU den Maschinenbefehl `0C 94 5D 00` aus?“** *Befehl holen, dekodieren, ausführen; Register und Programmzähler*
6. **„Wie sähe `if (digitalRead(2) && digitalRead(3))` als Schaltung aus – und was wäre daran anders als im Programm?“** *Boolesche Algebra, Gatter, Grenze zwischen Hardware und Software*
7. **„Wie kann ein 8-bit-Mikrocontroller 16-bit-Zahlen addieren?“** *Rechnen in mehreren Schritten, Carry-Flag, Aufbau der ALU*
8. **„Woher weiß der Controller nach einem Interrupt, wo er weitermachen muss?“** *Unterprogramme, Rücksprungadresse und Stack*

Der Unterschied zu einem „Arduino-Kurs“

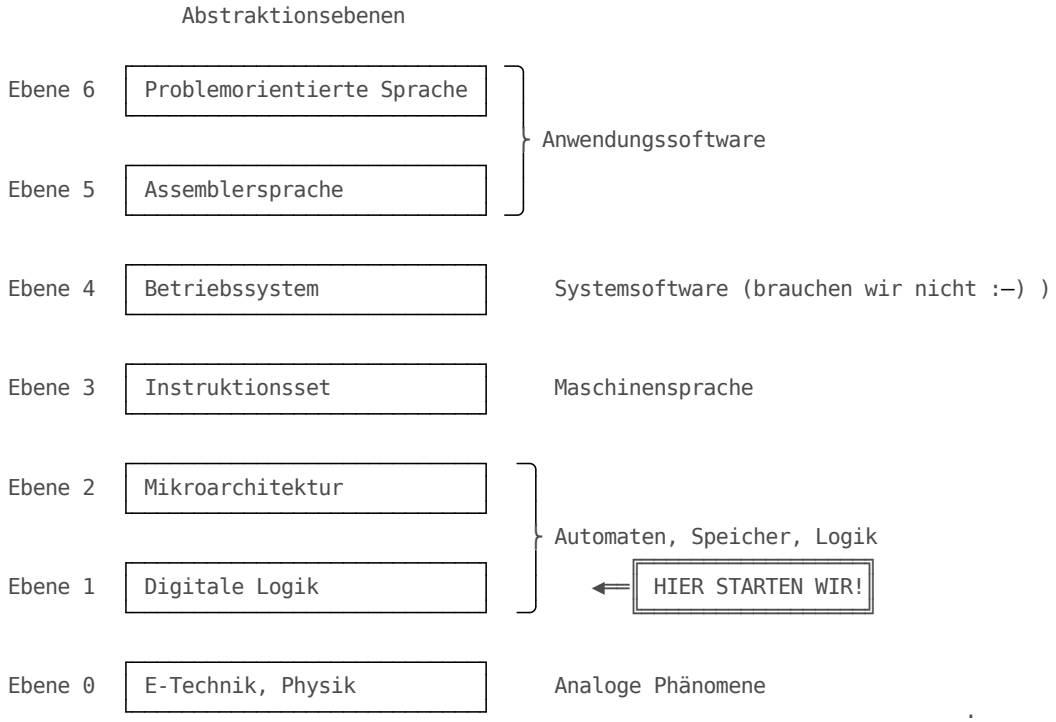
Nach einem typischen Arduino-Workshop können Sie LEDs blinken lassen. Nach diesem Kurs können Sie erklären, welche Register dabei beschrieben werden, warum digitalWrite deutlich mehr Befehle braucht als ein direkter Registerzugriff und wie aus dem Bit im Register ein Pegel am Pin wird. Das ist der Unterschied zwischen „es funktioniert“ und „ich verstehe, warum es funktioniert“.

Arduino-Kurs: „Drücke diesen Button, LEDs blinken“ ✨ *Magie!*

Unser Kurs: „Warum passiert das und wie funktioniert es bis hinunter zum Transistor?“

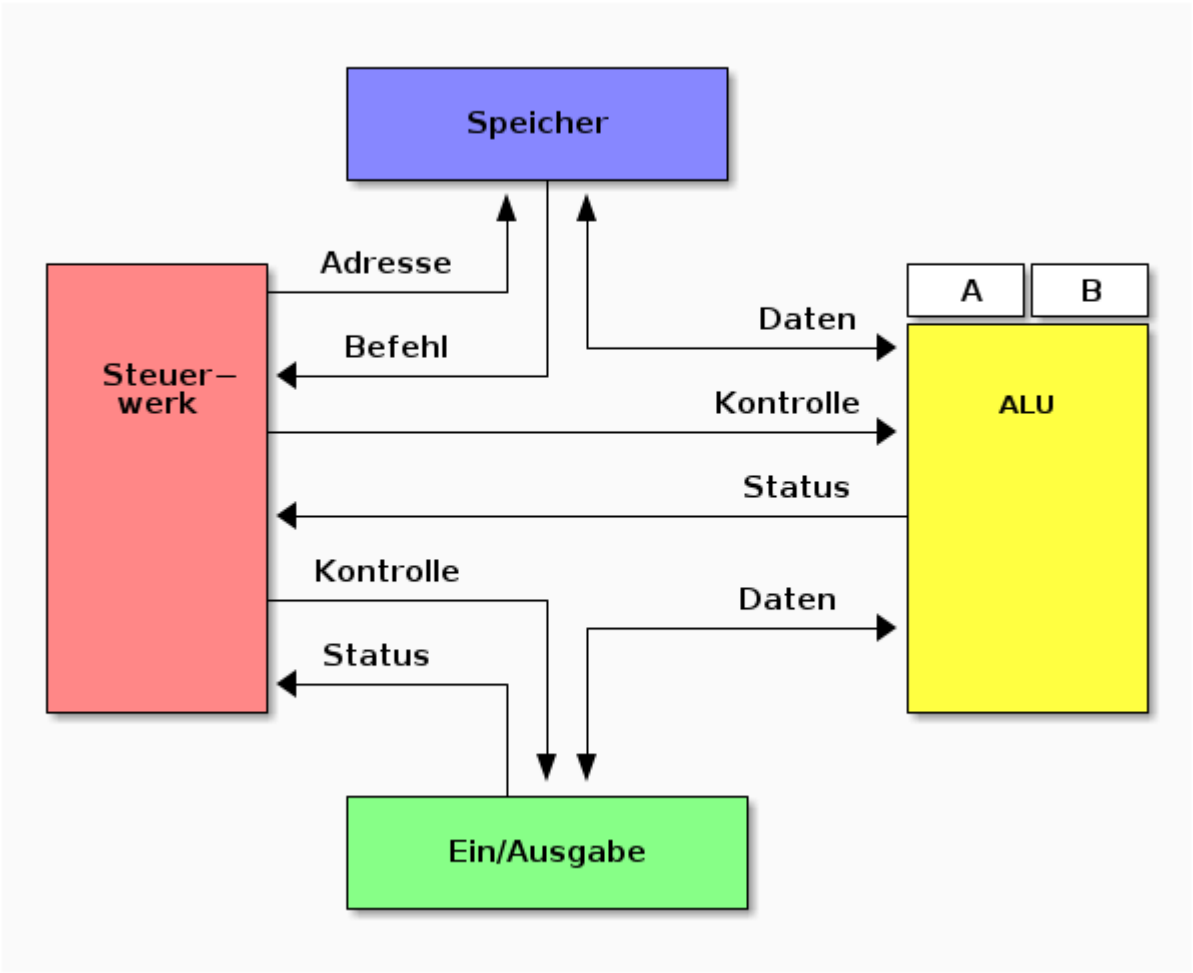
Unsere Bottom-Up-Reise

Schauen Sie sich diese Abstraktionsebenen an: Die meisten Programmierer arbeiten auf Ebene 6, nur wenige schauen auf Ebene 5 hinunter. Wir steigen bis auf Ebene 1 hinab und arbeiten uns systematisch nach oben.



Beispiel 1: Mikroarchitektur

Ein Rechner ist eine ziemlich komplizierte Maschine. Moderne CPUs oder GPUs umfassen Milliarden von Transistoren! Wir brauchen ein methodisches Verständnis und mehrere Abstraktionsebenen, um das zu verstehen.



Die Aufteilung in Steuerwerk, Rechenwerk, Speicher und Ein-/Ausgabe hat John von Neumann 1945 beschrieben, sie gilt bis heute – vom Mobiltelefon bis zum Arduino. Der ATmega328P auf dem Arduino weicht in einem Punkt ab: Programm und Daten liegen in getrennten Speichern. Man nennt das Harvard-Architektur; mehr dazu in Vorlesung 10 und 12.

Beispiel 2: Quer über alle Ebenen

Wir verfolgen jetzt eine einzige Arduino-Zeile durch die Abstraktionsebenen – so lange, bis wir beim Transistor ankommen, der die eigentliche Arbeit macht.

Wir bauen von den Grundlagen nach oben! Jede Ebene erklärt die nächste, bis Sie verstehen, wie Ihr `pinMode()` am Ende einen Transistor schaltet.

Schauen wir uns konkret an, was auf den einzelnen Ebenen passiert:

> **Sie müssen heute noch nicht alles verstehen.** Begriffe wie Interrupt, Register oder Flip-Flop sind heute nur Wegmarken – jeder davon bekommt im Semester seine eigene Vorlesung (Hinweise in Klammern).

Ebene 6 – Ihr Arduino-Code:

Ebene sechs sieht harmlos aus. Pin 13 ist übrigens kein Zufall – an ihm hängt die eingebaute LED der meisten Arduino-Boards.

```
pinMode(13, OUTPUT);  // Pin 13 als Ausgang konfigurieren
```

Ebene 6 – darunter: die Arduino-Bibliothek (C, gekürzt aus `wiring_digital.c`):

Auch die Arduino-Bibliothek ist in C geschrieben, also noch Ebene sechs. Sie übersetzt die Pin-Nummer in ein Register und ein Bit. Bevor sie das Register ändert, schaltet sie Interrupts kurz ab: Eine Interrupt-Routine könnte sonst zwischen Lesen und Zurückschreiben dasselbe Register verändern. Das nennt man eine Race Condition.

```
void pinMode(uint8_t pin, uint8_t mode) {
  uint8_t bit  = digitalPinToBitMask(pin);          // Pin 13 → Bit 5
  uint8_t port = digitalPinToPort(pin);             // Pin 13 → Port B
  volatile uint8_t *reg = portModeRegister(port);   // → &DDRB
  ...
  if (mode == OUTPUT) {
    uint8_t oldSREG = SREG;   // Statusregister sichern
    cli();                    // Interrupts aus
    *reg |= bit;              // DDRB |= (1 << 5)
    SREG = oldSREG;           // alten Zustand wiederherstellen
  }
}
```

Den Code müssen Sie nur überfliegen. Interrupts und Race Conditions behandeln wir in Vorlesung 14.

Kern des Ganzen ist eine einzige Zeile: `DDRB |= (1 << 5);` – setze Bit 5 im Register `DDRB`. Dabei schiebt `1 << 5` eine Eins um fünf Stellen nach links (`0000 0001` → `0010 0000`), und `|=` verknüpft das Register bitweise mit ODER: Bit 5 wird 1, alle anderen Bits bleiben, wie sie waren.

> Sie hören „Algorithmen und Datenstrukturen“ nicht parallel? Die Bitoperatoren von C erklärt zum Beispiel das Wikibook C-Programmierung.

Ebene 5 – Assembler:

Nur die Kernzeile `DDRB |= (1 << 5);` wird zu einem einzigen Befehl. Die ganze Funktion `pinMode` – mit Tabellen, Sichern des Statusregisters und Interrupt-Sperre – übersetzt der Compiler in deutlich mehr Befehle.

Schreiben wir die Kernzeile DDRB oder-gleich eins geschoben um fünf direkt, macht der Compiler daraus einen einzigen Assemblerbefehl: sbi, set bit in I/O register. 0x04 ist die Adresse von DDRB im Ein-/Ausgabebereich des Controllers. Dieses Programm können Sie direkt ausführen – es lässt die LED an Pin 13 blinken.



Simulation time: 00:35.280 (100%)

```
; Pin 13 (Port B, Bit 5) als Ausgang, dann blinken
#include <avr/io.h>

.global main
main:
  sbi   _SFR_IO_ADDR(DDRB), 5   ; DDRB |= (1 << 5)   →   sbi 0x04, 5

loop:
  sbi   _SFR_IO_ADDR(PINB), 5   ; eine 1 in PINB schaltet PORTB5 um
  rcall wait
  rjmp  loop

; 41 * 256 * 256 * 3 Takte ≈ 8.000.000 Takte = 0,5 s bei 16 MHz
wait:
  ldi   r20, 41
wait_outer:
  ldi   r19, 0                  ; 0 heißt hier: 256 Durchläufe (dec 0
    )
wait_middle:
  ldi   r18, 0
wait_inner:
  dec   r18
  brne  wait_inner
  dec   r19
  brne  wait_middle
  dec   r20
  brne  wait_outer
  ret
```

```
Sketch uses 160 bytes (0%) of program storage space. Maximum is 32256
bytes.
Global variables use 0 bytes (0%) of dynamic memory, leaving 2048 bytes
for local variables. Maximum is 2048 bytes.
```

Ebene 3 – Maschinencode:

Der Assembler übersetzt sbi 0x04, 5 in genau zwei Bytes: 25 und 9A. Bytes dieser Art – wenn auch nicht genau diese – stehen in der Hex-Datei vom Anfang. Wie das Steuerwerk aus diesen Bits erkennt, was zu tun ist, sehen wir in Vorlesung 11.

Assembler	Maschinencode im Speicher	Bedeutung der Bits
`sbi 0x04, 5`	`25 9A`	`1001 1010` = sbi · `00100` = Adresse 4 · `101` = Bit 5

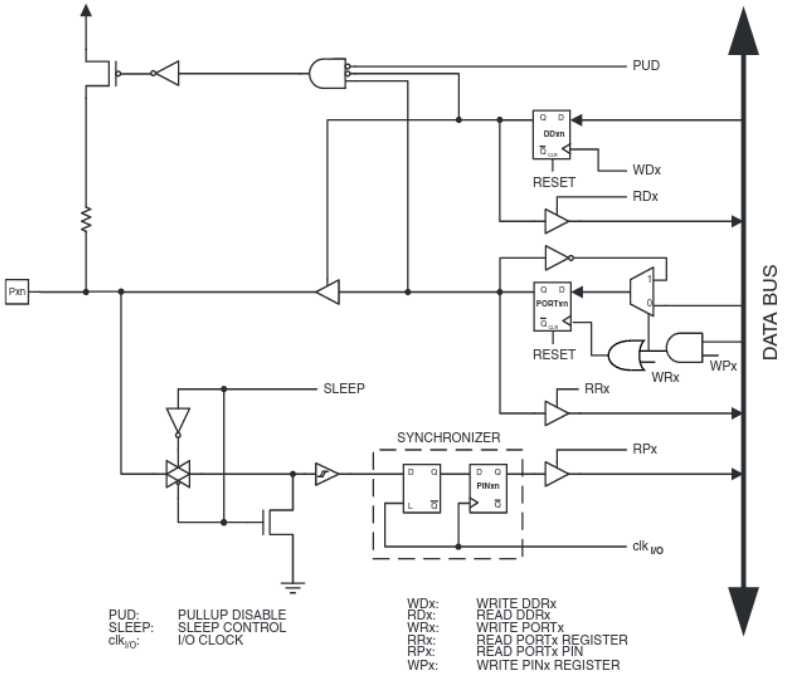
Der Befehl ist das 16-Bit-Wort `0x9A25`; im Speicher steht das niederwertige Byte zuerst.

Ebene 2 – Mikroarchitektur: Das Steuerwerk erkennt am Bitmuster `1001 1010` den Befehl `sbi`, leitet die Adresse 4 an den Ein-/Ausgabebereich weiter und setzt dort Bit 5. Wie ein Steuerwerk das macht, bauen wir in Vorlesung 10 und 11 selbst nach.

Ebene 1 – Hardware-Konfiguration:

Am Ziel: echte Hardware. Das Register DDRB besteht aus acht Flip-Flops, jedes gehört zu einem Pin. Ist Bit 5 gesetzt, wird der Ausgangstreiber von Pin 13 freigeschaltet: Ein Transistor kann den Pin mit der Versorgungsspannung verbinden, ein anderer mit Masse – das ist genau genommen schon Ebene 0. Das ist der Moment, in dem Software zu Physik wird.

Figure 14-2. General Digital I/O[1]



Aufbau eines Ein-/Ausgabe-Pins beim ATmega: oben das Flip-Flop DDxn (Richtung), in der Mitte PORTxn (Ausgabewert), unten der Eingang PINxn; Microchip, ATmega328P Datenblatt, General Digital I/O

Für Pin 13 ist im Bild x = B und n = 5: Das Flip-Flop DDxn ist also DDB5, Bit 5 im Register `DDRB`. Der Ausgangstreiber ist der dreieckige Verstärker in der Mitte, der den Pin Pxn treibt; der Transistor oben links gehört zum Pull-up-Widerstand.

HIGH und LOW sind Spannungen: Beim Arduino Uno liegt HIGH nahe der Versorgungsspannung von 5 V, LOW nahe 0 V, der Masse.

Das `DDRB`-Register steuert direkt die **Tri-State-Logik** des I/O-Pins:

- **DDRB[5] = 1**: Ausgangstreiber wird aktiviert → Pin kann HIGH/LOW ausgeben
- **DDRB[5] = 0**: Hochohmig → Pin kann Eingangssignale lesen; ist zusätzlich `PORTB[5] = 1` (das macht `INPUT_PULLUP`), zieht der Pull-up-Widerstand den Pin auf HIGH, solange ihn nichts auf LOW zieht

„Tri-State“ heißt: Ein Ausgang kennt drei Zustände – HIGH, LOW und „abgeschaltet“. Abgeschaltet (hochohmig) beeinflusst der Pin die angeschlossene Leitung praktisch nicht und kann so als Eingang dienen. Mehr dazu in Vorlesung 05 und 12.

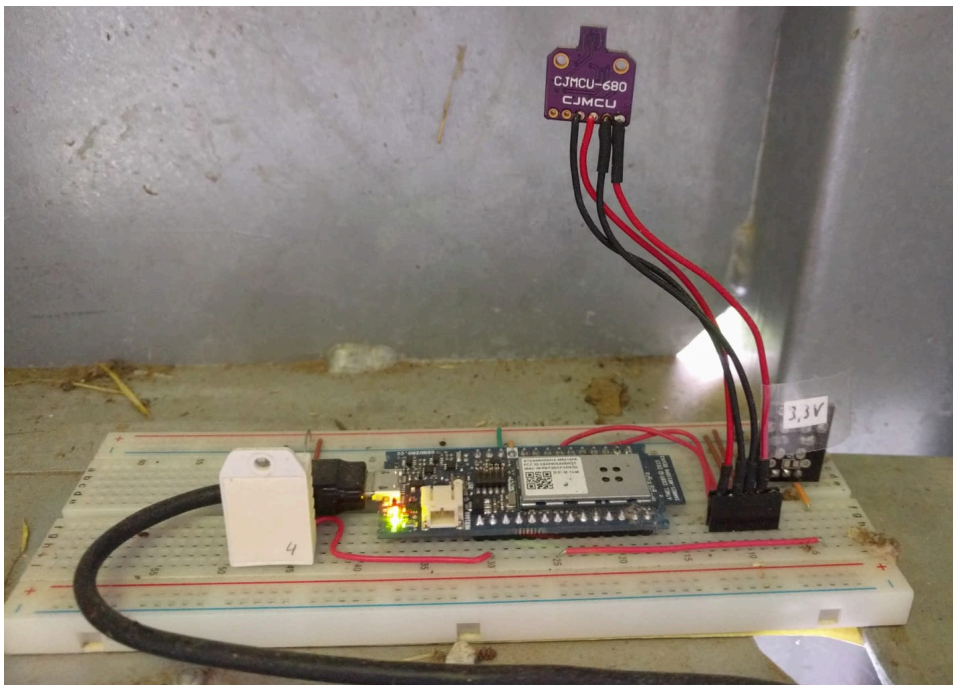
Ordnen Sie zu: Auf welcher Ebene liegt die jeweilige Darstellung?

Ebene 6	Ebene 5	Ebene 3	Eber	
◯	◯	◯	◯	`pinMode(13, OUTPUT);`
◯	◯	◯	◯	`DDRB |= (1 << 5);`
◯	◯	◯	◯	`sbi 0x04, 5`
◯	◯	◯	◯	`25 9A`
◯	◯	◯	◯	das Flip-Flop für Bit 5 in `DDRB`

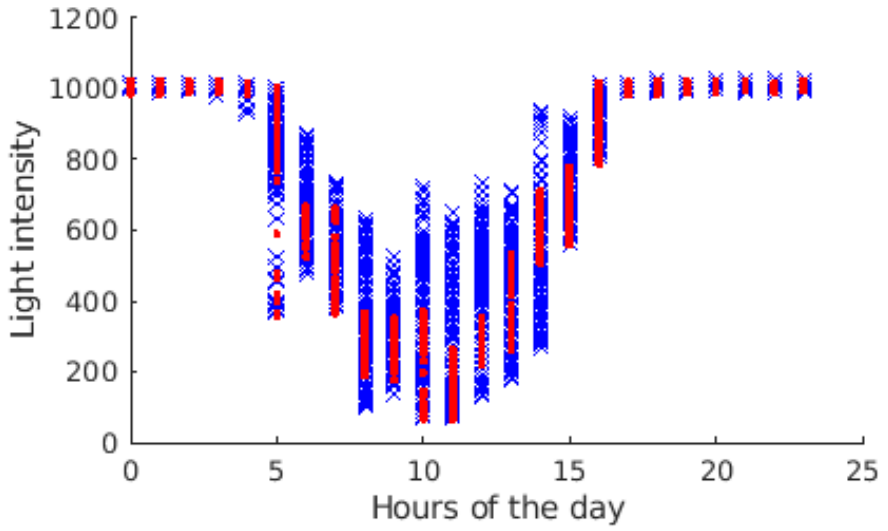
Was können Sie damit bauen?

Mit diesem Verständnis können Sie Systeme gezielt verbessern, statt nur Bibliotheken zu benutzen. Sie wissen, warum der Analog-Digital-Wandler manchmal „rauscht“, wie Sie Timer-Interrupts für präzises Timing einsetzen und warum manche Sensoren 5 V brauchen und andere mit 3,3 V auskommen.

Von einfachen LEDs zu echten Projekten:



Nach diesem Kurs wissen Sie nicht nur, wie man Sensordaten ausliest, sondern **warum** der 10-Bit-Analog-Digital-Wandler höchstens 1023 liefert und wie Timer-Interrupts die Abtastung steuern.



In diesem Aufbau bedeuten hohe Werte Dunkelheit und niedrige Werte Helligkeit – das liegt an der Beschaltung des Lichtsensors, nicht am Wandler. Sie verstehen, warum die Messwerte so aussehen, und können die Referenzspannung des Wandlers für präzisere Messungen wählen.

Wozu brauche ich das in meinem Studiengang?

Die Vorlesung hören Studierende aus sehr unterschiedlichen Richtungen. Jede Gruppe nimmt etwas anderes mit – aber alle profitieren davon, eine Schicht tiefer schauen zu können als bisher.

- **Robotik**: Jeder Roboter ist ein eingebettetes System – Sensoren lesen, Motoren ansteuern, rechtzeitig reagieren.
- **Angewandte Informatik**: Sie verstehen, was Ihr Programm auf der Hardware tatsächlich tut, und warum manches langsam oder speicherhungrig ist.
- **Mathematik**: Boolesche Algebra und Automaten werden hier zu Schaltungen – mit Laufzeiten, Takten und Grenzen der Physik.
- **Ingenieurwissenschaften**: Messen, Steuern und Regeln laufen heute fast immer auf einem Mikrocontroller. Abtastung, Quantisierung und Pulsweitenmodulation kennen Sie vielleicht aus der Messtechnik – hier sehen Sie, wie der Controller sie umsetzt (Vorlesung 13–15), bis hin zu Servo und Schrittmotor; in Vorlesung 12 wird auch ein Simulink-Modell zu Code für den Controller.

Organisation

Name	Email	Aufgabe
Prof. Dr. Sebastian Zug	sebastian.zug@informatik.tu-freiberg.de	Vorlesung, theoretische Übungsblätter
N.N.		praktische Übungen ab Januar

Bitte melden Sie sich im OPAL unter Digitale Systeme für die Veranstaltung an – dort heißt sie kurz „Digitale Systeme“. Dort finden Sie auch die aktuellen Informationen zur Veranstaltung.

Zeitplan

Die Veranstaltung wird sowohl für die Vorlesung als auch die Übung in Präsenz durchgeführt.

Veranstaltungen	Tag	Zeitslot	Ort	Bemerkung
Vorlesung I	Mittwoch	16:15 - 17:45	FOR-0270	wöchentlich
Vorlesung II	Donnerstag	11:30 - 13:00	FOR-0270	gerade Kalenderwoche
Übung, Gruppe 1	Montag	18:00 - 19:30	KKB-2097	wöchentlich ab Ende Novembe
Übung, Gruppe 2	Donnerstag	18:00 - 19:30	KKB-2097	wöchentlich ab Ende Novembe

Die Übungen starten Ende November mit theoretischen Übungsblättern und werden wöchentlich durchgeführt. Ab Januar vertiefen Sie Ihre Kenntnisse in praktischen Übungen an Mikrocontrollerschaltungen.

Ergänzend stehen Remote-Labore bereit, um einzelne Aspekte der Lehrveranstaltung zu vertiefen.

Prüfungsmodalitäten

Prüfungsform: Die Veranstaltung wird mit einer schriftlichen Prüfung abgeschlossen. Diese wird als **Open Book Klausur** entworfen. Sie dürfen dazu alle schriftlichen, nicht-digitalen Materialien verwenden.

Eine druckbare PDF-Fassung jeder Vorlesung finden Sie in der Kursübersicht – ideal, um eigene Notizen für die Klausur direkt daneben zu schreiben.

Im Laufe der Übungen werden wir „alte“ Übungsaufgaben durchspielen, die Ihnen einen Eindruck von der Prüfung vermitteln sollen.

Literaturempfehlungen

1. Umfassende Lehrbücher
    - David A. Patterson, John L. Hennessy: Computer Organization & Design
    - B. Becker, R. Drechsler, P. Molitor: Technische Informatik - Eine Einführung, Pearson Studium, 2005
    - Hoffmann, D. W.: Grundlagen der technischen Informatik, Hanser Verlag
2. Videos
    - Youtube – „How a CPU Works“ Link

Bei den jeweiligen Vorlesungsinhalten werden zusätzliche Materialien angegeben.

Ihre Vorkenntnisse

Die Vorlesung hören Studierende mit sehr unterschiedlichen Vorkenntnissen. Ihre Antworten helfen uns, Beispiele und Übungen passend zu wählen. Die Abfrage ist anonym; wir sammeln die Antworten im Hörsaal über das LiaScript-Klassenzimmer.

Aus welchem Studiengang kommen Sie?

- ◯ Angewandte Informatik
- ◯ Robotik
- ◯ Mathematik
- ◯ ein Ingenieurstudiengang
- ◯ ein anderer Studiengang

Welche Erfahrung haben Sie mit Arduino oder anderen Mikrocontrollern?

- ◯ keine Vorerfahrung
- ◯ kurz ausprobiert, Praktikum oder Ähnliches
- ◯ umfangreiche „Basteleien“
- ◯ Berufserfahrung mit Mikrocontrollern

Können Sie in C programmieren?

- ◯ ich lerne C in diesem Semester (Algorithmen und Datenstrukturen)
- ◯ ja, schon vorher
- ◯ nein, und ich höre auch keine C-Vorlesung

Haben Sie Zugriff auf einen Arduino Uno, Nano oder einen anderen ATmega-Mikrocontroller?

- ◯ ja, ein Controller ist vorhanden
- ◯ nein

Engagement und Motivation

Credit-Points: 6

Arbeitsaufwand: Der Zeitaufwand beträgt 180 h. Davon sind rund **45 h Präsenzzeit** – pro Woche im Mittel 1,5 × 90 Minuten Vorlesung und 0,5 × 90 Minuten Übung – und rund **135 h Selbststudium**. Das Selbststudium umfasst die Vor- und Nachbereitung der Lehrveranstaltung, die eigenständige Lösung von Übungsaufgaben sowie die Prüfungsvorbereitung.

Open Educational Resources

Die Lehrmaterialien sind offen verfügbar und liegen auf GitHub:

https://github.com/TUBAF-IfI-LiaScript/VL_EingebetteteSysteme

Sie sind in LiaScript geschrieben, einer Erweiterung von Markdown. Markdown beschreibt Texte mit wenigen, gut lesbaren Zeichen – `# Überschrift`, `*kursiv*`, `**fett**`, `+ Aufzählung`. LiaScript ergänzt, was in der Lehre fehlt: ausführbaren Code, Simulationen, Quizze und Animationen. Den Simulator haben Sie oben schon benutzt.

Zum Weiterlesen: Markdown Guide · LiaScript-Dokumentation · Einführungsvideos

Eine Übersicht über alle Begriffe und roten Fäden der Vorlesung bietet die Kurslandkarte. Sie wächst mit dem Semester: Am Ende jeder Vorlesung fragen wir Sie nach den Begriffen des Tages.

Trotz Simulation und Virtuellem …

… braucht es aber auch immer etwas zum Anfassen. Der Simulator zeigt viel – aber er verzeiht auch Dinge, die echte Hardware nicht verzeiht: falsche Verdrahtung, zu hohe Ströme, prellende Taster. Ab Januar arbeiten Sie deshalb in der Übung mit echten Mikrocontrollerschaltungen.

Generelles Engagement

- Stellen Sie Fragen, seien Sie kommunikativ!
- Organisieren Sie sich in Arbeitsgruppen!
- Bringen Sie sich mit Implementierungen als Vortragende in die Veranstaltung ein.

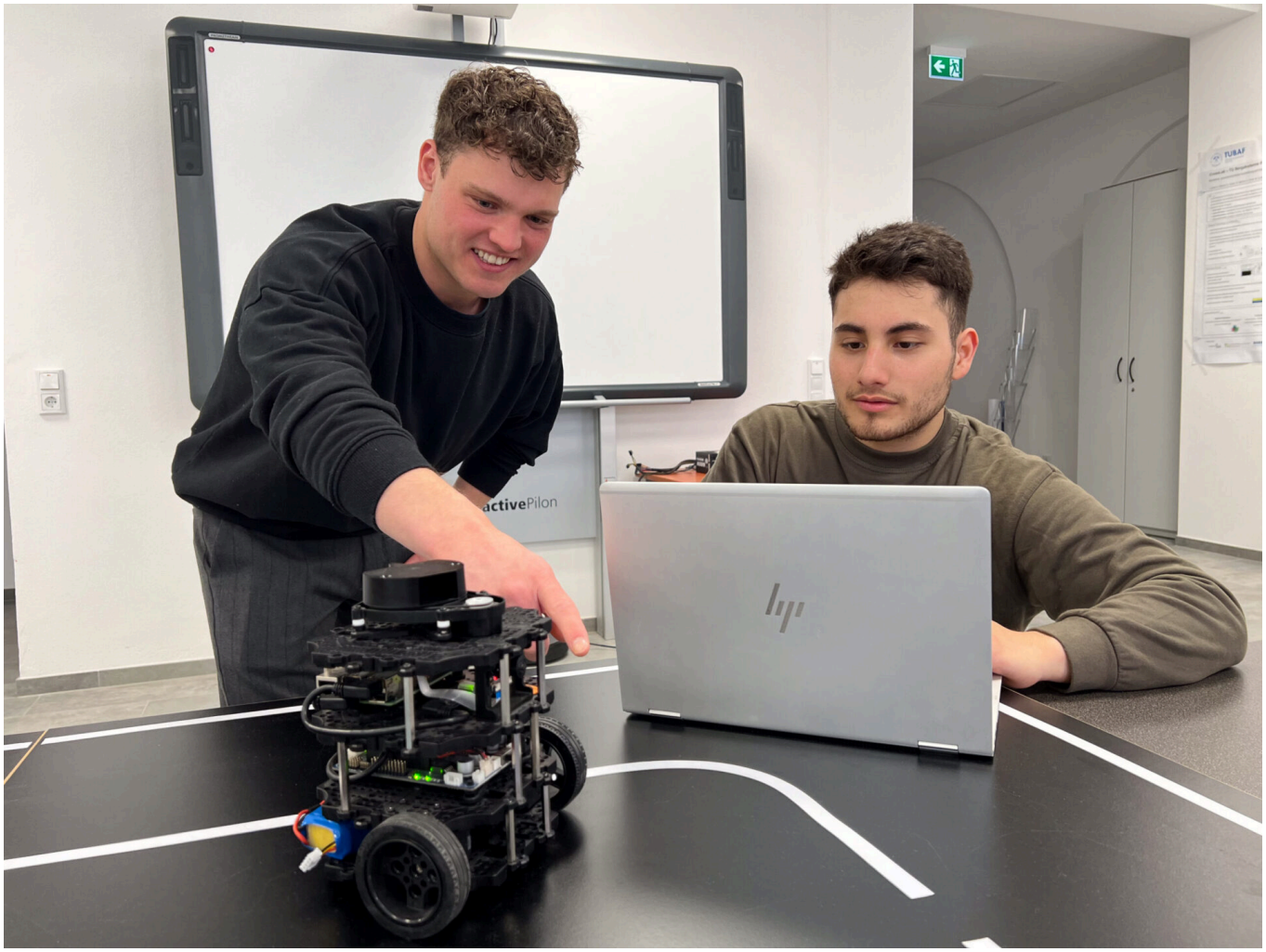
Und wenn Sie dann immer noch programmieren wollen …

Dann wartet das **racetech** Team auf Sie … autonomes Fahren im Formula Student Kontext.



Schauen Sie im RoboLab vorbei!

Lion und Caio warten auf Sie …



Was Sie heute mitnehmen

- Ein **eingebettetes System** ist ein Rechner mit fester Aufgabe in einem Gerät – mit Sensoren, Aktoren, knappen Ressourcen und oft Echtzeitanforderungen.
- Zwischen `pinMode(13, OUTPUT)` und dem Pegel an Pin 13 liegen **Abstraktionsebenen**: Hochsprache, Assembler, Maschinencode, Mikroarchitektur, digitale Logik, Physik.
- Am Ende steht eine einfache Bitoperation – `sbi 0x04, 5` setzt ein Flip-Flop in `DDRB` –, und die Hex-Datei ist nichts anderes als solche Befehle hintereinander.
- Wir gehen **von unten nach oben**: Nächste Woche beginnen wir mit der Geschichte und der Logik, im Januar sind wir beim Controller angekommen.

Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester, und Sie bestimmen mit, was hineinkommt. Ihre Antworten sammeln wir im Hörsaal gemeinsam über das LiaScript-Klassenzimmer; nach der Vorlesung werden die Begriffe dieser Woche in der Karte freigeschaltet.

Welche Begriffe aus der heutigen Vorlesung gehören für Sie in die Kurslandkarte?

Texteingabe ...

Wo begegnet Ihnen das, was heute dran war – in Ihrem Studiengang oder in Geräten Ihres Alltags?

Texteingabe ...

Hausaufgabe

- Legen Sie sich einen GitHub-Account an … und finden Sie als Erste oder Erster einen Tippfehler in den Unterlagen, den Sie als Contributor korrigieren 🙂
- Organisieren Sie sich in einer gemeinsamen Chatgruppe! Bevor Sie heute den Raum verlassen, sollten Sie dort Mitglied sein.
- Lassen Sie die LED im Assembler-Beispiel doppelt so schnell blinken. Es gibt mehr als einen Weg – finden Sie zwei und begründen Sie, welcher der Verdopplung näher kommt und warum keiner sie exakt trifft. Die Auflösung gibt es zu Beginn der nächsten Vorlesung.

Nächste Woche

Am **Mittwoch, 28.10.**, reisen wir durch 400 Jahre Rechentechnik – von Zahnrädern über Relais und Röhren bis zum Mikroprozessor –, am **Donnerstag, 29.10.**, beginnt mit der Booleschen Algebra die Digitaltechnik.